Continued Interactive GSLTs and the Cost Endofunctor

A construction one level up from cost-accounted Rholang

(revised: wrapping by construction)

L. G. Meredith* *with* Claude (Anthropic)

F1R3FLY.io

May 2026

Abstract

We lift the cost-accounting construction of the rho calculus from a single calculus to an endofunctor on a category of interactive graph-structured lambda theories (GSLTs). The earlier work folded phlogiston accounting into the grammar of Rholang by signing terms and replacing the communication rule by a family of rules that rewrite *signed* terms while consuming *tokens*. Here we observe that this is a generic procedure governed by a single shape of dynamics, the *symmetric metered cut*: an interaction constructor K brings two (co-)introductions $K_{\mathrm{p}}(I, C_{\mathrm{p}})$ and $K_{\mathrm{e}}(J, C_{\mathrm{e}})$ into contact, and a contraction $\mathsf{compute}(I, J, C_{\mathrm{p}}, C_{\mathrm{e}})$ produces residual continuations. An interactive GSLT whose dynamics admits this presentation, together with a computable section of its structural-congruence quotient and a contraction compatible with wrapping, we call a *continued* interactive GSLT. The point of the adjective is not that such a theory is already metered or wrapped — a continued interactive GSLT is an ordinary, un-metered calculus — but that it carries exactly the structure needed for the cost construction to *build* a metered version from it.

That construction is the endofunctor $\mathfrak{C}$. Given a continued interactive GSLT, $\mathfrak{C}$ produces a calculus in which the continuation slots are *wrapped terms by construction*: every continuation is a metered thunk in the grammar of the image. The decisive design move is that this wrapping lives in the *output*, so that "every redex sits inside a wrapper" is a sorting invariant of $\mathfrak{C}(G)$ preserved by reduction — not a property that must be re-established by traversing each contractum. The source theory's contraction need not be lifted; in the image it already maps wrapped terms to wrapped terms, and the only run-time interaction with tokens is *unwrapping*: forcing a thunk by spending a matching token. Metering is thereby *lazy* — work is charged when performed, not when exposed — and the temporal token stack is consumed exactly in step with the reductions, realising the modulus of the underlying cut elimination as the run length itself.

We show that $\mathfrak{C}$ is an endofunctor on the category **ciGSLT** of continued interactive GSLTs with bisimulation-preserving, quote-faithful morphisms. Throughout we keep **ciGSLT** formally distinct from the ambient category **iGSLT** of interactive GSLTs: a forgetful functor U : **ciGSLT** $\to$ **iGSLT** exhibits "continued" as a proper structural enrichment of "interactive," so that the endofunctor's domain pins down exactly which interactive theories admit sound, lazy cost accounting. Finally, iterated cost accounting flattens — nested signatures multiply and stacks-of-stacks concatenate — so $\mathfrak{C}$ is a (non-idempotent) monad, resolving through two adjunctions with opposite embeddings: a structural free–forgetful adjunction that detaches the apparatus, and, over a Turing-complete base, an internalisation adjunction that dissolves the apparatus into the base's own computation up to weak bisimulation.

*lgreg.meredith@f1r3fly.io

Contents

1 Introduction

In previous work we showed that the cost-accounting layer of Rholang — the phlogiston (phlo) system — need not be a privileged runtime extension. By treating signatures as a species of name, token stacks as first-class terms, and the for/send operators as ranging over *signed* terms, we folded cost accounting directly into the grammar and operational semantics of the rho calculus. The single communication rule Comm became a family of rules that gate communication on the consumption of a matching token. Because the rho calculus is the formal core of Rholang, this revised calculus serves as the design specification for rearchitecting phlo accounting in Rholang 1.4.

This note takes the construction *one level up*, asking what the move requires of the underlying theory and to which theories it applies. Two observations drive the development. First, the dynamics must be presented as a *symmetric* cut: the eliminand is not a bare term but a co-introduction carrying its own continuation, exactly as the sender $x!(U)$ in rho carries a payload U with the same

status as the receiver's continuation T. Second — and this is the heart of the revision — a theory presented in this form carries enough structure for the cost endofunctor to build a metered version in which the continuation slots are *wrapped terms by construction*. We are careful to keep the two apart: the underlying theory (a *continued* interactive GSLT) is an ordinary un-metered calculus that merely *admits* the construction; the wrapping is a feature of the *image* $\mathfrak{C}(G)$, not of G. In that image every continuation is a metered thunk in the grammar, so the contractum is born guarded: no traversal re-wraps it, the contraction is not lifted, and the absence of leaks is a sorting invariant of $\mathfrak{C}(G)$ rather than a dynamic obligation. The run-time semantics of the metered calculus only ever *unwraps*, forcing a thunk by spending a token.

The λ-calculus is the foil throughout. In the rho calculus several distinct structures are fused into $|$ and $@\cdot$; pulling them apart in λ, where they are visibly separate, exposes the construction as a genuine endofunctor and identifies precisely what an interactive theory must supply.

Section 2 isolates the symmetric metered cut. Section 3 introduces wrapping by construction, derives no-leak as subject reduction, and contrasts lazy with eager metering. Section 4 identifies the spatial monoid K and the temporal monoid $:$, with the stack as a reified clock and lazily extracted modulus. Section 5 treats signature construction as a section of the structural-congruence quotient, with de Bruijn canonicalisation supplying it for λ. Section 6 assembles the endofunctor $\mathfrak{C}$ on **ciGSLT**. Section 7 records the graded adequacy statement. Section 8 works the two anchoring instances. Section 9 observes that iterated cost accounting flattens, so that $\mathfrak{C}$ is a monad, and exhibits its two resolutions: a structural free–forgetful adjunction and, over Turing-complete bases, a computational internalisation adjunction.

2 The symmetric metered cut

2.1 Interactive GSLTs

A *graph-structured lambda theory* is a multi-sorted term theory $G = (\Sigma, E, R)$: a signature Σ of sorts and operators, a set E of equations imposing a structural congruence $\equiv$, and a set R of rewrite rules. It is *interactive* when it carries a distinguished interacting sort P with a labelled transition system, generated by R modulo $\equiv$, on which bisimulation is the intended equivalence. The rho calculus ($P =$ processes, $\equiv$ the $|$-monoid congruence with α, $R =$ Comm) and the λ-calculus ($P =$ terms, $\equiv = \alpha$, $R = \beta$) are the motivating examples.

Definition 2.1 (The category **iGSLT**)**. iGSLT** is the category whose objects are interactive GSLTs and whose morphisms are *theory maps* — sort- and operator-preserving translations that respect $\equiv$ and R — that are **bisimulation-preserving** on the interacting sort.

This is the base category of the present note. The cost construction does *not* act on all of **iGSLT**; it acts on the objects that carry enough additional structure to be metered, which we isolate as a subcategory in Section 6. Keeping the two apart is deliberate: "interactive" is the ambient setting, while "continued" (Definition 6.1) is the structural condition under which sound, lazy cost accounting exists. The distinction is what the endofunctor's domain makes precise — an interactive GSLT is a place where processes interact; a continued interactive GSLT is one whose interaction is presented as a meterable cut.

2.2 The symmetric presentation

The cost construction acts on dynamics presented in a particular shape. Write the generating rule of R as

$$\frac{(C'_{\mathrm{p}},\, C'_{\mathrm{e}}) \;=\; \mathsf{compute}(I,\, J,\, C_{\mathrm{p}},\, C_{\mathrm{e}})}{K\big(K_{\mathrm{p}}(I,\, C_{\mathrm{p}}),\; K_{\mathrm{e}}(J,\, C_{\mathrm{e}})\big) \;\longrightarrow\; K'\big(C'_{\mathrm{p}},\, C'_{\mathrm{e}}\big)} \tag{$\dagger$}$$

distinguishing:

the interaction constructor K, which brings two operands into *contact*. In rho, $K = |$; in λ, $K = \mathrm{App}$.

two (co-)introductions $K_{\mathrm{p}}(I, C_{\mathrm{p}})$ and $K_{\mathrm{e}}(J, C_{\mathrm{e}})$, each a constructor separating a bound or active part $(I,\, J)$ from a *continuation* $(C_{\mathrm{p}},\, C_{\mathrm{e}})$. In rho, $K_{\mathrm{p}} = \mathrm{for}$ with $I \sim (y, x)$ and $C_{\mathrm{p}} = T$, while $K_{\mathrm{e}} = !$ with $J \sim x$ and $C_{\mathrm{e}} = U$.

the contraction $\mathsf{compute}$**,** a partial operation that, on contact, transforms the two continuations into residuals $C'_{\mathrm{p}}, C'_{\mathrm{e}}$ repackaged by K'. In rho and λ it is (semantic) substitution.

We call ($\dagger$) a *cut*: two introductions meet and a contraction eliminates them. The form is *symmetric* in the two operands: the eliminand is not a bare term but a co-introduction with its own continuation. This matches rho exactly, where $x!(U)$ was never bare — its payload U has the same status as the receiver's continuation T. The earlier asymmetric form, with a bare eliminand E, was an under-elaboration; the λ-calculus recovers it as the *degenerate* case in which K_{e} has an empty bound part, so that the argument is its own continuation (Section 8).

Remark 2.2 (K is the only constructor that needs overloading)**.** The cut isolates one contact site. Every cost rule interposes a token between the operands of K and drains it on contact. There is one K per cut, hence one constructor to make polymorphic. "Meter the cut" is the generic content of "place valuable friction at every interaction."

3 Wrapping by construction

This section describes what the cost endofunctor *builds*. Given a continued interactive GSLT G — an un-metered calculus presented as a symmetric cut, with a section and a wrapping-compatible contraction (Section 6) — the endofunctor $\mathfrak{C}$ produces a metered calculus $\mathfrak{C}(G)$ whose grammar makes every continuation a metered thunk. The wrapping discipline below is therefore a property of the image $\mathfrak{C}(G)$; the source G supplies only the structure that makes the discipline installable.

3.1 Continuations are metered thunks in the image

In $\mathfrak{C}(G)$ the cost endofunctor adjoins a sort Sig of signatures (Section 5), a *wrapped-term* sort $\mathbb{T}$ with the wrapper $\{\cdot\}_s : P \times \mathsf{Sig} \to \mathbb{T}$, and a sort Stk of token stacks

$$S \;::=\; (\,) \;\mid\; s : S, \qquad s \in \mathsf{Sig}. \tag{1}$$

The pivotal discipline is grammatical, and it is imposed by $\mathfrak{C}$ on its image: **in $\mathfrak{C}(G)$ every continuation slot of K_{p} and K_{e} has sort $\mathbb{T}$.** Structural composition (parallel composition in rho) lives at the $\mathbb{T}$ level, between wrappers, so that each redex-bearing thread is individually wrapped. Thus a continuation in the metered calculus is not a bare process awaiting metering; it is a *metered thunk* — $\{P\}_s$ — whose activation is gated by a token keyed to s, and whose own continuations are again thunks. (In the source G these slots carried ordinary terms; re-sorting them to $\mathbb{T}$ is part of

what $\mathfrak{C}$ does, Construction 6.6.) Reading $C_{\mathrm{p}} = \{P\}_{s_{\mathrm{p}}}$ and $C_{\mathrm{e}} = \{Q\}_{s_{\mathrm{e}}}$, the contraction of the image has the type

$$\mathsf{compute} \;:\; \mathit{Bnd} \times \mathit{Bnd} \times \mathbb{T} \times \mathbb{T} \;\longrightarrow\; \mathbb{T} \times \mathbb{T},$$

mapping wrapped continuations to wrapped continuations. It need not know that wrapping exists: substituting a wrapped term into a position yields a wrapped term, so $\mathsf{compute}$ preserves the sort $\mathbb{T}$ for free.

3.2 The gated rule: force by unwrapping

A wrapped redex is forced by consuming a matching token. The single-signature rule is

$$\frac{(C'_{\mathrm{p}}, C'_{\mathrm{e}}) = \mathsf{compute}(I, J, C_{\mathrm{p}}, C_{\mathrm{e}})}{K\big(\{K(K_{\mathrm{p}}(I, C_{\mathrm{p}}),\, K_{\mathrm{e}}(J, C_{\mathrm{e}}))\}_s,\; s : S\big) \;\longrightarrow\; K\big(K'(C'_{\mathrm{p}}, C'_{\mathrm{e}}),\; S\big)} \tag{R1}$$

in which the outer polymorphic K combines the wrapped redex with the token stack, the head cell s is consumed (the act of *unwrapping* the redex), and the result $K'(C'_{\mathrm{p}}, C'_{\mathrm{e}})$ is — by the typing of $\mathsf{compute}$ — already built from wrapped terms. There is no re-wrapping step and no lifted contraction. The remaining token-distribution cases vary how the redex is split across K and how tokens are presented:

$$K\big(K(\{K_{\mathrm{p}}(I, C_{\mathrm{p}})\}_{s_1},\, \{K_{\mathrm{e}}(J, C_{\mathrm{e}})\}_{s_2}),\; s_1 : s_2 : S\big) \;\longrightarrow\; K\big(K'(C'_{\mathrm{p}}, C'_{\mathrm{e}}),\; S\big), \tag{R2}$$

$$K\big(K(\{K_{\mathrm{p}}(I, C_{\mathrm{p}})\}_{s_1},\, \{K_{\mathrm{e}}(J, C_{\mathrm{e}})\}_{s_2}),\; K(s_1 : S_1,\, s_2 : S_2)\big) \;\longrightarrow\; K\big(K'(C'_{\mathrm{p}}, C'_{\mathrm{e}}),\; K(S_1, S_2)\big), \tag{R3}$$

each under the same premise. Here the continuations $C_{\mathrm{p}}, C_{\mathrm{e}}$ are themselves wrapped (suppressed inner signatures), so the residuals $C'_{\mathrm{p}}, C'_{\mathrm{e}}$ remain wrapped and the next layer of redexes is guarded without any action by the rule.

3.3 No leak, as a sorting invariant

Definition 3.1 (Well-wrapped configuration)**.** A configuration of $\mathfrak{C}(G)$ is *well-wrapped* when it is well-sorted in the grammar of Construction 6.6 — in particular, every continuation occupies a slot of sort $\mathbb{T}$, so every redex occurrence lies inside a wrapper $\{\cdot\}_s$.

Lemma 3.2 (Subject reduction for wrapping)**.** *Well-wrapping is preserved by* (R1)–(R3)*.*

Proof sketch. Each rule replaces a wrapped redex by $K'(C'_{\mathrm{p}}, C'_{\mathrm{e}})$ with $(C'_{\mathrm{p}}, C'_{\mathrm{e}}) = \mathsf{compute}(\dots)$. Since $\mathsf{compute} : \cdots \to \mathbb{T} \times \mathbb{T}$ and K' packages terms of sort $\mathbb{T}$, the residual is well-sorted; the consumed cell leaves a well-sorted stack. Redexes elsewhere are untouched. Hence the result is well-wrapped. □

Corollary 3.3 (No leak, by construction)**.** *In a well-wrapped configuration no redex can fire without being unwrapped, i.e. without consuming a token. There is no dynamic wrapping operation; the cost-accounting steps only ever unwrap.*

The contrast with the earlier formulation is the substance of this revision.

Remark 3.4 (Eager vs. lazy metering)**.** An alternative discipline re-wraps the contractum at contraction time, traversing it to sign each exposed redex — charging *eagerly* for every redex the step exposes. Wrapping by construction charges *lazily*: each redex is born wrapped and is charged only when actually forced. Lazy metering pays for work performed, not work merely exposed; redexes beneath a discarded binder are never charged. The token stack drains exactly in step with the reductions performed, which is also the operational reality of phlogiston consumption.

Remark 3.5 (Duplication needs no fresh signatures)**.** Because multiplicity is carried by the *stack* (linear token consumption) and not by key distinctness, copying a wrapped term $\{Q\}_s$ copies its key s but not its tokens. Each copy must be forced separately and so funded separately from the stack: n copies of an s-redex require n cells keyed to s. Shared keys with a linear token supply already make duplication cost, so no fresh-per-copy minting is required. Created redexes — where a substituted wrapped term lands in head position — are guarded for the same reason: the substituted material carries its own wrapper. Thus a single mechanism (wrapped continuations + linear stack) handles duplicated and created redexes alike, where the eager formulation needed a separate fresh-signature discipline.

4 Two monoids: space and time

The construction places two combination operators side by side, of different character. The interaction constructor K is *spatial*: it records what is in contact with what, and may carry equations — associative–commutative in rho (an unordered bag of processes), free in λ (a positional application spine). The cons operator : of (1) is *temporal*: it records what is spent next, then next. It is a free monoid (a list), *never* commutative, because reduction is sequential even when interaction is parallel. Each forcing pops a cell; the order of popping is the order of steps. The stack is the time axis of the computation reified as a term: $\equiv$ leaves it invariant (spatial congruence consumes no time), $\longrightarrow$ pops it (a step is a tick), and its ordering is the arrow of the computation.

Proposition 4.1 (Stack consumption is the modulus, lazily realised)**.** *For a terminating reduction in $\mathfrak{C}(G)$, the number of cells consumed equals the number of forced redexes, which equals the number of cuts eliminated — including those introduced by duplication, each charged when forced. The consumed prefix of the temporal stack is therefore an exact operational modulus for the cut elimination, realised by running the reduction rather than by a separate traversal.*

Where eager metering pre-computes a bound by traversing each contractum, wrapping by construction lets the reduction *be* the bound extraction: the consumed stack is the realised cost, tight by construction and lazy. Evaluation is cut elimination; the token stack is the extracted modulus; metering at the granularity of forcing makes the two coincide.

5 Signature construction as a section of the quotient

Tokenisation derives its expressiveness from minting signatures from runtime data, so the construction adjoins a constructor turning terms into signatures. We isolate what the base theory must provide.

5.1 Quotient and section

Let $\mathcal{T}$ be the free (pre-congruence) syntax of the interacting sort and $\widehat{\mathcal{T}} = \mathcal{T}/\equiv$ its quotient by structural congruence. Two maps relate them, differing precisely in their relationship to $\equiv$:

- the **quotient** $\mathcal{T} \twoheadrightarrow \widehat{\mathcal{T}}$, *respecting* $\equiv$, invertible up to congruence — the structure a name constructor exposes when channels are taken up to $\equiv$ (in rho, $@P$ with $@P = @Q \iff P \equiv Q$). It is not a constructor the cost functor requires; it is the congruence the cut is gated modulo.
- a **section** $\mathsf{cf} : \widehat{\mathcal{T}} \to \mathcal{T}$, a choice of canonical representative per class. Composing with a collision-resistant digest gives the signature constructor $\# = \mathrm{digest} \circ \mathsf{cf} : \widehat{\mathcal{T}} \to \mathsf{Sig}$, a one-way commitment to a representative. It does *not* respect $\equiv$ — and must not, since a commitment identifying congruent-but-distinct representatives would be no commitment.

Sig carries a commutative monoid $(\mathsf{Sig}, *, (\,))$ compounding signatures, matching the compound-signature gating of (R2), (R3). Signatures never reduce: they are a rewrite-free sub-theory meeting the behavioural theory only at the matching guard. Hence the $\equiv$-incoherence of $\#$ is harmless for bisimulation, which factors as "P-bisimulation gated by signature-key matching."

Remark 5.1 (The two axes, disentangled)**.** In rho the section and the communication-quotient are the *same* operator $@\cdot$ viewed two ways, which is why the cost story there appeared to need "two axes of reflection." They are not two signature schemes. One axis ($@\cdot$ up to $\equiv$) is the communication quotient the LTS already carries; the other ($\# = \text{digest} \circ \mathsf{cf}$) is the authentication section the cost functor adds. The signature is $\#$ alone; the functor needs both because metering tracks *what a process does* (quotient) and *who committed to it* (section).

5.2 The λ-calculus supplies a section without a channel constructor

The λ-calculus has no analogue of $@\cdot$. This is the sharp test, and it passes, because the functor never required a *channel* constructor; it required a *section*. For λ the congruence is α and a section is a canonical representative of each α-class: the de Bruijn encoding. Thus $\#_\lambda = \text{digest} \circ (\text{de Bruijn})$, and a signature commits to the de Bruijn form. The communication-quotient role that $@\cdot$ played in rho is, in λ, filled not by a constructor but by α-equality of redexes in the guard. Rho fused section and quotient; λ pulls them apart, and the construction is identical.

Definition 5.2 (Section-equipped)**.** An interactive GSLT is *section-equipped* when it comes with a computable section $\mathsf{cf} : \widehat{\mathcal{T}} \to \mathcal{T}$ of its $\equiv$-quotient (equivalently, a computable canonical-form function on the interacting sort).

6 The cost endofunctor

6.1 Continued interactive GSLTs

Definition 6.1 (Continued interactive GSLT)**.** A *continued interactive GSLT* is an interactive GSLT (an object of **iGSLT**) *equipped with* the following additional structure:

(i) a presentation of its dynamics in symmetric metered-cut form (†), naming the constructors $K, K_\mathrm{p}, K_\mathrm{e}, K'$ and the partial contraction $\mathsf{compute}$;

(ii) a section (Definition 5.2), i.e. it is section-equipped; and

(iii) *wrappability*: the contraction $\mathsf{compute}$ is well-sorted as an operation on wrapped continuations, $\mathsf{compute} : \cdots \to \mathbb{T} \times \mathbb{T}$ — equivalently, $\mathsf{compute}$ is definable on metered thunks, mapping wrapped inputs to wrapped outputs.

We write **ciGSLT** for the category whose objects are continued interactive GSLTs and whose morphisms are theory maps that are **bisimulation-preserving** on the interacting sort and **quote-faithful**: injective on the reachable signature keys.

The adjective *continued* thus names a strict enrichment of *interactive*: clauses (i)–(iii) are data and conditions added on top of an **iGSLT**-object, not a restriction of it. Forgetting them recovers the underlying interactive GSLT.

Definition 6.2 (The forgetful functor)**.** Let $U : \mathbf{ciGSLT} \to \mathbf{iGSLT}$ send a continued interactive GSLT to its underlying interactive GSLT — discarding the metered-cut presentation, the section, and the wrappability witness — and act as the identity on the underlying theory map of a morphism.

Proposition 6.3 (The distinction is proper)**.** *U is faithful but neither full nor essentially surjective. Hence* **ciGSLT** *is a genuine, proper subcategory of* **iGSLT***: strictly more structure on objects, strictly fewer morphisms between them.*

Discussion. Faithful, not full. U forgets no information about a morphism's action, so it is faithful. It is not full because a **ciGSLT**-morphism must additionally be quote-faithful: an **iGSLT**-morphism that is bisimulation-preserving yet collapses two distinct signature keys is a map between the underlying objects with no lift to **ciGSLT**. (This refines the naive expectation that the inclusion be full: the section structure on objects induces a real constraint on morphisms, so the inclusion is faithful-not-full rather than full.)

Not essentially surjective. Not every interactive GSLT admits the added structure. Clause (i) requires the dynamics to factor as a symmetric cut — contact K then contraction $\mathsf{compute}$ — which a theory with non-local or unfactorable rewrites need not; clause (ii) requires a *computable* section of the $\equiv$-quotient, which a theory with undecidable structural congruence lacks; clause (iii) requires the contraction to preserve wrapping. An interactive GSLT failing any of these is an object of **iGSLT** with no preimage under U. □

Remark 6.4 (λ exhibits the distinction)**.** The λ-calculus is an object of **iGSLT** *as such*: terms, α, β. It becomes an object of **ciGSLT** only once one *chooses* the added structure — the de Bruijn section for (ii), the degenerate-K_{e} symmetric-cut presentation of β for (i), substitution-on-thunks for (iii) (Section 8). The same underlying calculus thus sits at both levels, witnessing that "continued" is added structure and not a property read off the interactive GSLT alone. This is exactly the role λ plays as a control: each clause is a real question about it, with a real answer.

The morphism conditions are dual and jointly necessary. Bisimulation-preservation forbids distinguishing too much (behaviourally); quote-faithfulness forbids collapsing too much (representationally). A morphism merging two signatures would let a term signed by one drain a token minted for the other, so the target would gain transitions the source lacked. $\mathfrak{C}$ lives exactly where both are jointly satisfiable.

Remark 6.5 (Wrappability replaces traversability)**.** The condition (iii) is the wrapped-by-construction successor of the earlier "traversable contractum" requirement. We no longer need to walk a contractum to re-sign its exposed redexes; we need only that the contraction be a well-sorted operation on thunks. An opaque contraction that destroyed wrapping — returning bare redexes — would violate (iii) and is excluded, which is the same exclusion as a contraction whose cost could not be accounted.

6.2 The functor on objects

Construction 6.6 ($\mathfrak{C}$ on objects)**.** For $G = (\Sigma, E, R, K, K_{\mathrm{p}}, K_{\mathrm{e}}, K', \mathsf{compute}, \mathsf{cf}) \in$ **ciGSLT**, let $\mathfrak{C}(G)$ be the continued interactive GSLT with:

- **signature** Σ extended by Sig (with $\# = \mathrm{digest} \circ \mathsf{cf}$, monoid $*, (\,)$); the wrapped-term sort $\mathbb{T}$ with $\{\cdot\}_s$; the token-stack sort Stk with $:$; the polymorphic typing of K over wrapped-term/stack and stack/stack pairs; and, crucially, the re-sorting of every continuation slot of $K_{\mathrm{p}}, K_{\mathrm{e}}$ to $\mathbb{T}$;
- **equations** E with the commutative-monoid equations on $(\mathsf{Sig}, *, (\,))$ and the inertness of Sig and Stk under $\longrightarrow$;
- **rewrites** the gated family (R1)–(R3), with the *unchanged* contraction $\mathsf{compute}$ (now read at sort $\mathbb{T} \times \mathbb{T}$);

- **section** the evident lift of cf ignoring inert sorts.

Proposition 6.7 (Closure)**.** $\mathfrak{C}(G)$ *is again a continued interactive GSLT.*

Proof sketch. The gated rules are themselves in symmetric metered-cut form: the outer polymorphic K is contact, the signed redex and the token stack are the two operands, and $\mathsf{compute}$ (with the stack tail) is the contraction. $\mathfrak{C}(G)$ is section-equipped by the lifted cf, and wrappable because $\mathsf{compute}$ preserves $\mathbb{T}$ by Lemma 3.2. All three clauses of Definition 6.1 hold. □

6.3 The functor on morphisms

Construction 6.8 ($\mathfrak{C}$ on morphisms)**.** For $f : G \to H$ in **ciGSLT**, let $\mathfrak{C}(f)$ act as f on the interacting sort and transport the adjoined structure along f: signatures by $\#_H \circ f$ on canonical forms, wrappers and stacks componentwise, the polymorphic K structurally.

Theorem 6.9 ($\mathfrak{C}$ is an endofunctor on **ciGSLT**)**.** $\mathfrak{C} : \mathbf{ciGSLT} \to \mathbf{ciGSLT}$ *preserves identities and composition, and* $\mathfrak{C}(f)$ *is a* **ciGSLT***-morphism whenever* f *is.*

Proof sketch. Object closure is Proposition 6.7; identity and composition preservation are immediate from the componentwise definition. By the factoring of bisimulation as "P-bisimulation gated by key matching," and since f is P-bisimulation-preserving and quote-faithful, $\mathfrak{C}(f)$ preserves a gated transition iff f preserves the underlying P-transition and respects the key match — both hold. Quote-faithfulness transports along the injective $\#_H$. □

7 Graded adequacy

The token stack grades transitions of $\mathfrak{C}(G)$ by the signature monoid $(\mathsf{Sig}, *, (\,))$: a forced step is labelled by the signature(s) it consumes. Applying OSLF — which auto-generates a Hennessy–Milner logic from a GSLT with an adequacy theorem — to the graded transition system of $\mathfrak{C}(G)$ yields a *graded* Hennessy–Milner logic with modalities $\langle a \rangle_s$ indexed by the consumed signature.

Theorem 7.1 (Graded adequacy, schematic)**.** *For* $\mathfrak{C}(G)$ *the graded Hennessy–Milner logic generated by OSLF is adequate for quote-faithful bisimulation: two configurations satisfy the same graded-HML formulae iff they are quote-faithfully bisimilar.*

This also explains the throughput improvement mechanically. In the translation presentation, fuel acquisition was an encoded multi-step protocol on auxiliary channels; under $\mathfrak{C}$ it is a single primitive graded transition — the grading absorbs the sub-protocol into the step relation (*protocol fusion*). With wrapping by construction the fusion is total: there is no re-wrapping sub-step either, only forcing. Together with Proposition 4.1, phlogiston is the conserved grade, invariant under $\equiv$ (signatures inert), strictly consumed along $\longrightarrow$ (forcing drains the temporal stack), in the order recorded by :.

8 Two anchoring instances

8.1 Communication (rho): the symmetric case

Instantiate at the rho calculus: $K = |$ (associative–commutative); $K_{\mathrm{p}} =$ for with $I \sim (y, x)$ and continuation $C_{\mathrm{p}} = T = \{P\}_{s_{\mathrm{p}}}$; $K_{\mathrm{e}} = !$ with $J \sim x$ and continuation $C_{\mathrm{e}} = U = \{Q\}_{s_{\mathrm{e}}}$;

compute semantic substitution; cf a normal form for the |-congruence. Both operands are genuine introductions carrying wrapped continuations — this is the symmetric cut in full. Rule (R1) reads

$$\{\mathrm{for}(y \leftarrow x)\, T \mid x!(U)\}_s \mid s : S \longrightarrow T\{@U/y\} \mid S,$$

and since $T = \{P\}_{s_\mathrm{p}}$ and $U = \{Q\}_{s_\mathrm{e}}$, the residual $T\{@U/y\} = \{P\{@\{Q\}_{s_\mathrm{e}}/y\}\}_{s_\mathrm{p}}$ is again a wrapped thunk, to be forced later by a token keyed to s_p. No re-wrapping occurs; the AC structure of | supplies the split-redex and split/combined-token cases (R2), (R3) up to $\equiv$, recovering the earlier five-rule lattice.

8.2 β-reduction: the degenerate case

Instantiate at the λ-calculus: $K = \mathrm{App}$ (free, no equations); $K_\mathrm{p} = \lambda$ with $I = x$ and wrapped body $C_\mathrm{p} = \{M\}_{s_\mathrm{p}}$. The argument is not a binding co-introduction, so K_e is *degenerate*: empty bound part, the argument being its own wrapped continuation $C_\mathrm{e} = \{N\}_{s_\mathrm{e}}$. The contraction is substitution, $\mathsf{compute}(x, -, \{M\}_{s_\mathrm{p}}, \{N\}_{s_\mathrm{e}}) = (\{M\}_{s_\mathrm{p}}\{\{N\}_{s_\mathrm{e}}/x\},\ (\,))$. Rule (R1) reads

$$\mathrm{App}\big(\{\mathrm{App}(\lambda x.\{M\}_{s_\mathrm{p}},\ \{N\}_{s_\mathrm{e}})\}_s,\ s : S\big) \longrightarrow \mathrm{App}\big(\{M\}_{s_\mathrm{p}}\{\{N\}_{s_\mathrm{e}}/x\},\ S\big),$$

with $s = \#_\lambda$ of the de Bruijn form of the redex. Every copy of $\{N\}_{s_\mathrm{e}}$ that substitution places into M carries its wrapper, so duplicated redexes are guarded; and where x stood in head position, the landed $\{N\}_{s_\mathrm{e}}$ guards the newly created application — both by construction, with no traversal (Remark 3.5). Token supply is *not* function application; the temptation to write $(\{\mathrm{App}(\lambda x.M, N)\}_s\ s)$ feeds the key as a value and gates nothing. The token sits at the contact site and is consumed by forcing.

9 The cost monad and two adjunctions

Three further facts organise the construction: iterated cost accounting flattens, so $\mathfrak{C}$ carries a monad structure; and that monad relates to its base through two different adjunctions — one structural, one computational — whose embeddings have opposite characteristics.

9.1 Flattening: $\mathfrak{C}$ is a monad

Applying the construction twice yields $\mathfrak{C}^2(G) = \mathfrak{C}(\mathfrak{C}(G))$: a calculus that meters the metering of G. Its signatures are signatures of signed terms, its stacks are stacks of stacks, and its wrappers nest as $\{\{P\}_{s_2}\}_{s_1}$. There is a canonical way to collapse the two layers into one, and — tellingly — it is assembled entirely from the two monoids already present in the construction.

Construction 9.1 (Unit and multiplication)**.** Define the *unit* $\eta_G : G \to \mathfrak{C}(G)$ to wrap each term with the unit signature and present it against the freely available unit token,

$$\eta_G(P) \;=\; \{P\}_{(\,)}.$$

Since $(\,)$ is the unit of $(\mathsf{Sig}, *, (\,))$, an $(\,)$-keyed redex fires without net resource; thus η_G embeds G as the *cost-free fragment* of $\mathfrak{C}(G)$ — the metered calculus restricted to trivially-funded interactions. (This is consistent with the source/image discipline of Section 3: η is precisely the wrapping that installs apparatus at zero charge.)

Define the *multiplication* $\mu_G : \mathfrak{C}^2(G) \to \mathfrak{C}(G)$ to flatten the two layers by

(i) **multiplying nested signatures** in Sig, $\{\{P\}_{s_2}\}_{s_1} \mapsto \{P\}_{\,s_1 * s_2}$; and

(ii) **concatenating the stack-of-stacks** into a single stack — the free-monoid (list) multiplication on Stk.

The outer account is thereby merged into the inner: the grade of the flattened redex is the product of the two grades, and its temporal stack is the concatenation, in the order the two layers would have drained them.

Proposition 9.2 (The cost monad)**.** $(\mathfrak{C}, \eta, \mu)$ *is a monad on* **ciGSLT**. *Its laws descend from the laws of the two constituent monoids: the unit laws from* $s * (\,) = s = (\,) * s$ *together with the singleton/empty laws of the list monad, and associativity from associativity of* $*$ *and of list concatenation.*

Proof sketch. η and μ are **ciGSLT**-morphisms. Each is quote-faithful, since $*$ is injective in each argument up to the collision-resistance of the digest, and bisimulation-preserving, since flattening neither creates nor destroys gated transitions — it relabels each by the product grade and consolidates the (spatially commutative, temporally sequential) consumption order consistently. Naturality is the componentwise action on the adjoined sorts. The monad identities $\mu \circ \mathfrak{C}\eta = \mathrm{id} = \mu \circ \eta_{\mathfrak{C}}$ reduce to the unit laws of $*$ and of concatenation; $\mu \circ \mathfrak{C}\mu = \mu \circ \mu_{\mathfrak{C}}$ reduces to their associativity. □

Remark 9.3 (A genuine, non-idempotent resource monad)**.** $\mathfrak{C}$ is not idempotent: μ_G is no isomorphism, because concatenating two stacks forgets the boundary between them and multiplying two grades forgets the factorisation. Flattening is a true *merge* of two resource accounts, not the collapse of a redundant copy. This is the expected signature of a resource monad — re-metering a metered calculus yields a finer account, which μ then consolidates — and it distinguishes $\mathfrak{C}$ from idempotent "closure" monads.

The Eilenberg–Moore algebras of $\mathfrak{C}$ are continued interactive GSLTs equipped with a coherent *discharge* map $\alpha : \mathfrak{C}(A) \to A$ interpreting tokens — a calculus that knows how to "pay" — and the Kleisli category is that of cost-accounted translations between calculi. The structural adjunction below is the free resolution generating the monad; the Turing-complete adjunction is a separate phenomenon.

9.2 Adjunction I: install ⊣ forget

Let **Cost-ciGSLT** be the category of *cost-accounted* continued interactive GSLTs: objects carry the installed apparatus (Sig, Stk, the wrapper, the gated family), morphisms preserve it. A forgetful functor $\mathsf{Forget} : \textbf{Cost-ciGSLT} \to \textbf{ciGSLT}$ strips the apparatus, returning the underlying un-metered calculus; a free functor $\mathsf{Free} : \textbf{ciGSLT} \to \textbf{Cost-ciGSLT}$ installs it — exactly Construction 6.6.

Proposition 9.4 (Free–forgetful resolution)**.** $\mathsf{Free} \dashv \mathsf{Forget}$*, with unit* η *of Construction 9.1, and the induced monad* $\mathsf{Forget} \circ \mathsf{Free}$ *on* **ciGSLT** *is* $\mathfrak{C}$*.*

The embedding is *structural and strict*: Free adjoins sorts and rules, Forget erases them on the nose. Crucially, forgetting is *not* behaviour-preserving in the metering sense — erasing the apparatus removes the gating, so $\mathsf{Forget}\,\mathsf{Free}\,G$ runs G's interactions *without friction*. The structural resolution answers "what apparatus was added?" and lets us remove it; it does not claim the metered and un-metered dynamics agree. Its character is **structure-preserving, behaviour-altering**.

9.3 Adjunction II: internalise ⊣ include, over a universal base

Restrict to the full subcategory $\mathbf{ciGSLT}_{\mathrm{TC}} \hookrightarrow \mathbf{ciGSLT}$ of *Turing-complete* continued interactive GSLTs. For such a G, the entire metering apparatus of $\mathfrak{C}(G)$ — signature equality, stack maintenance, gating — is computable, hence *encodable in G itself.* This yields an *internalisation*

$$\mathcal{I}_G : \mathfrak{C}(G) \longrightarrow G,$$

realising the cost-accounted semantics inside the un-metered but universal base by simulation: tokens become encoded data, the gated rules become an interpreter loop, and each forced step is simulated by a finite run of G.

Proposition 9.5 (Internalisation as an adjoint retraction)**.** *For $G \in \mathbf{ciGSLT}_{\mathrm{TC}}$ the internalisation satisfies $\mathcal{I}_G \circ \eta_G \approx \mathrm{id}_G$ up to weak bisimulation, exhibiting $\mathfrak{C}(G)$ as a behavioural retract of G. In the bicategory of continued interactive GSLTs, simulations, and simulation transformations, $\eta_G \dashv \mathcal{I}_G$ is an adjoint retraction: the unit η_G is an (iso-up-to-bisimulation) section and the counit is the collapsing simulation $\eta_G \circ \mathcal{I}_G \Rightarrow \mathrm{id}_{\mathfrak{C}(G)}$.*

Proof sketch. Turing completeness of G gives a computable encoding of the finite data and decidable guards of $\mathfrak{C}(G)$; standard interpreter constructions make $\mathcal{I}_G$ a weak simulation that reflects every gated transition. Composing with η_G (which installs apparatus at unit cost) returns G's dynamics up to the administrative reductions of the interpreter, i.e. up to weak bisimulation. The triangle identities hold up to the 2-cells witnessing these weak bisimulations, which is the content of an adjunction internal to the simulation bicategory. □

The embedding is the mirror image of Adjunction I. It is *behaviour-preserving* — the simulation faithfully reproduces every gated step — but *structure-encoding rather than structure-preserving*: the clean sorts $\mathsf{Sig}, \mathsf{Stk}$ dissolve into encoded data, and the agreement holds only up to weak bisimulation, since the interpreter interposes administrative reductions. Its character is **behaviour-preserving, structure-dissolving**, exactly opposite to Adjunction I.

Remark 9.6 (Two senses of conservativity, and where they hold)**.** The two adjunctions express two senses in which cost accounting is conservative over its base. Structurally (Adjunction I, available for every object) the apparatus is a detachable layer. Computationally (Adjunction II) the apparatus is already expressible, so metering is a definitional extension up to weak bisimulation — but this holds *only* when the base is Turing complete. The second is therefore a genuine property of the object: universal interactive theories are exactly those whose cost-accounted versions fold back into them, a distinction invisible to the purely structural resolution.

The cost-accounting move of the rho calculus is an instance of a generic endofunctor on the category **ciGSLT** of *continued* interactive GSLTs: interactive theories presented as a symmetric metered cut (contact K, two (co-)introductions $K_{\mathrm{p}}(I, C_{\mathrm{p}})$, $K_{\mathrm{e}}(J, C_{\mathrm{e}})$, contraction $\mathsf{compute}$), equipped with a computable section of their structural-congruence quotient, and with continuation slots sorted as *wrapped terms by construction.* The functor $\mathfrak{C}$ signs terms, adjoins a token stack, makes the contact constructor K polymorphic, and replaces the cut by a gated family. Its soundness is a sorting invariant: every redex is born inside a wrapper, reduction preserves wrapping (Lemma 3.2), so no computation proceeds for free and the contraction need not be lifted — the cost-accounting steps only ever unwrap. Metering is thereby lazy, paying for work performed rather than exposed, and the temporal token stack is consumed exactly in step with reduction, realising the modulus of cut elimination as the run length. The cons operator is extension in time, dual to the spatial extension of K; OSLF lifts $\mathfrak{C}(G)$ to a graded Hennessy–Milner logic with adequacy "graded-HML

equivalence equals quote-faithful bisimulation." The λ-calculus, with K and the section visibly distinct and K_{e} degenerate, shows the construction requires a section — de Bruijn canonicalisation — not a channel constructor; rho's apparent need for two axes of reflection was the self-overlap of one operator serving as both quotient and section.

Finally, the construction is deliberately sited one rung below the ambient category: $\mathfrak{C}$ is an endofunctor on **ciGSLT**, not on **iGSLT**, and the forgetful functor U : **ciGSLT** $\to$ **iGSLT** (Definition 6.2, Proposition 6.3) records that "continued" is a proper structural enrichment of "interactive." The content of the note is therefore as much about *where* cost accounting lives as about what it does: an interactive GSLT is a setting for interaction; a continued interactive GSLT is one whose interaction is presented as a meterable cut, and it is precisely on these that the cost endofunctor exists — indeed, as a monad (Section 9), since iterated metering flattens. Its two resolutions sharpen the picture: the structural free–forgetful adjunction shows the apparatus is detachable, while over a Turing-complete base the internalisation adjunction shows it is already expressible, so that cost accounting is a definitional extension up to weak bisimulation. That second adjunction holds only for universal bases, and so quietly begins to sort interactive theories by a structural property — a thread we leave for the sequel.